

- JS4-数组
 - 4 数组降维
 - 输入描述:
 - 输出描述:
 - 4.1 普通递归实现
 - 4.2 reduce实现
 - 4.3 Array.some + 扩展运算符
 - 4.4 split + toString
 - 4.5 正则 + JSON实现
 - 4.6 flat

JS4-数组

4 数组降维

牛客题霸 JS20 数组扁平化

请补全JavaScript代码，要求将数组参数中的多维数组扩展为一维数组并返回该数组。注意：1. 数组参数中仅包含数组类型和数字类型

输入描述:

```
[1,[2,[3,[4]]]]
```

输出描述:

```
[1,2,3,4]
```

4.1 普通递归实现

普通递归的思路很容易理解，就是通过 `for` 循环的方式，逐层逐个元素地去展平，如果当前元素是一个数组，那么就对它进行递归处理，再将递归处理的结果拼接到结果数组上。

```
function flatten1(arr){
  let res = [];
  for (let i=0;i<arr.length;i++){
    if(Array.isArray(arr[i])){
      res = res.concat(flatten1(arr[i]));
    }else{
      res.push(arr[i])
    }
  }
  return res;
}
```

4.2 reduce实现

`reduce`是JS数组中非常强大的一个方法，同时也是JS中的一个函数式编程API。

上面的递归实现的关键就是对数组的每一项进行处理，遇到数组就递归处理它，既然需要循环和结果数组，那么我们可以使用`reduce`来简化我们的代码：

```
let arr = [1, [2, [3, 4]]];
function flatten(arr) {
  return arr.reduce(function(pre, cur){
    return pre.concat(Array.isArray(cur) ? flatten(cur) : cur)
  }, [])
}
console.log(flatten(arr));// [1, 2, 3, 4, 5]
```

4.3 Array.some + 扩展运算符

扩展运算符是ES6的新特性之一，用它操作数组可以直接展开数组的第一层，利用这个特性，我们可以不使用递归来实现数组的展平，这是因为每一次递归都是对当前层次数组的一次展开，而扩展操作符就是干这工作的。

```
function flatten2(arr){
  let res = [...arr];
  while(res.some(Array.isArray)){
    res = [].concat(...res);
  }
  return res;
}
```

代码中使用了数组的另一个方法 `some`，目的是判断当前数组是否还有数组元素，如果有则对数组进行一层展开，同时将展开结果作为下一次判断的条件，这样就像剥洋葱一样，一层层地剥开洋葱皮，当循环条件不满足时说明数组里已经没有数组元素了，这是数组已经完全扁平。

4.4 split + toString

我们也可以通过 `split` 和 `toString` 两个方法，来共同实现数组扁平化。`toString` 是 `Object` 原型链上的一个方法，由于 JS 中所有对象都派生自 `Object` 对象，所以它们都能调用 `toString`，只不过不同的对象可能会对这个方法进行改写以输出自己想要的格式。数组的 `toString` 方法会将数组转换成一个元素间以逗号相隔的字符串，它内部会先将数组展平成一维后再转换成字符串，因此我们可以先利用 `toString` 进行展平，然后再通过 `split` 方法以逗号分隔每个元素来复原一个包含所有元素的数组，从而实现数组的扁平化。

```
function flatten3(arr){
  let res = arr.toString().split(',').map((val)=>{return Number(val)});
  return res;
}
```

不过需要注意的是，虽然这个方法非常简单，但具有一定的局限性，对于包含引用类型元素的数组来说，在 `toString` 过程中会发生类型转换，从而使得转换结果异常，因为对于引用类型转成字符串，会调用引用类型的 `toString`，上面提到不同对象会对它进行改写，例如函数就会得到一个函数体的代码字符串，而不是我们想要的函数引用。

因此，使用这种方式我们要看元素类型而定。

4.5 正则 + JSON实现

正则实现也是先将数组转换成字符串的表达形式，这里将数组转成字符串我们使用 `JSON.stringify` 方法，将数组转换成一个由括号包裹、元素间以逗号相隔的字符串，例如 `"[1, [2, [3, [4, 5]]], 6]"`，从转换后的字符串可以发现，每一对括号包着的范围就是一个层级，数组的展平就是将内部的层级全部消除，只保留最外层，在这里，就是将括号全部替换成空字符，得到一个元素以逗号相隔字符串，最后将利用 `JSON.parse` 方法解析成数组对象。

```
function flatten5(arr){
  let str = JSON.stringify(arr);
  str = str.replace(/(\[|\])/g, '');
  // 拼接最外层，变成JSON能解析的格式
  str = '[' + str + ']';
  return JSON.parse(str);
}
```

同样，这种方式在处理引用数据类型上也有局限性，同时还要注意元素是否是JSON的合法数据类型。

4.6 flat

`Array.prototype.flat`是ES6新增的一个数组方法，它的作用就是用来数组扁平化，并且根据传入的参数来决定展开的层级，是数组扁平化的终极解决方案。

```
function flatten4(arr){
  return arr.flat(Infinity);
}
```

作者：JaylenL

链接：<https://juejin.cn/post/6998879961935249422>

来源：稀土掘金

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。